Allocation dynamique et mémoire

Driss MATROUF
Professeur des universités
Avignon Université

La pile et le tas

- **La mémoire est une succession d'octets (8 bits), organisés les uns à la suite des autres et directement accessibles par une adresse.**
- **En C/C++, la mémoire pour stocker des variables est organisée en deux catégories :**
  - La pile (stack)
  - Le tas (heap)
- **Attention, d'autres zones mémoire permettant de ranger d'autres informations telles que les programmes eux mêmes, les variables globales, existent.**
- **Dans les langages de programmation compilés, la pile est l'endroit où sont stockés les paramètres d'appel et les variables locales des fonctions**

```
bool distance(Point A, Point B)
{
        double r ;
        ....
}
```

Dans ce cas A, B et r sont stockés dans la pile

La pile (Stack)

- **La pile est un espace mémoire réservé au stockage des variables locales. Elles seront désallouées automatiquement.**

  ....
  {
  Point A ;
  ...
  } // ici disparition de A (libération de A)(désallocation de A)

- **Sa taille est limitée mais on peut la régler (ulimit -a sous ubuntu)**

  {
  int T[1000000] ;
  .....
  }
  à éviter !!!!

```
matrouf@DESKTOP-H6U0D74:~$ ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority             (-e) 0
file size               (blocks, -f) unlimited
pending signals                 (-i) 7823
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files                      (-n) 1024
pipe size            (512 bytes, -p) 8
POSIX message queues     (bytes, -q) 819200
real-time priority              (-r) 0
stack size              (kbytes, -s) 8192
cpu time               (seconds, -t) unlimited
max user processes              (-u) 7823
virtual memory          (kbytes, -v) unlimited
file locks                      (-x) unlimited
matrouf@DESKTOP-H6U0D74:~$
```

- **La pile utilise la politique LIFO (last in first out) ce qui signifie « dernier entré premier sorti ».**
  - Lorsqu'un objet est dépilé il est effacé de la mémoire

Le tas (heap)

- **Le tas est l'autre segment de la mémoire utilisé lors de l'allocation dynamique de la mémoire durant l'exécution du programme.**
- **Sa taille est souvent considérée comme illimitée mais elle est en réalité limitée**
  - Attention au swap
- **Les opérateurs du langage C++ new et delete permettent respectivement d'allouer et désallouer la mémoire sur le tas.**
- **Contrairement à la mémoire dans la pile, la mémoire allouée dans le tas doit être désallouée explicitement.**
- **La taille de la pile limitée (ici 8Mo), Il est évidemment recommandé d'allouer dans le tas les « grosses » variables sous peine de surprises !**

Adresse d’une variable : un nouveau type

- Toute variable est accessible grâce à son adresse : son emplacement en mémoire, c’est un entier indiquant le numéro d’octet ou commence la variable.
- L’opérateur & appliqué à une variable donne une valeur : l’adresse de la variable

```
{
    int a ;
    cout << & a<<endl ;
} // affiche : 0x7ffff5a3b524
```

- 0x7ffff5a3b524 en entier vaut « 140736805945860 » est l’adresse de la variable *a*
- &a est une valeur de type adresse d’un entier, on dit aussi de type pointeur sur un entier.
- Le nouveau type est noté : int *, car la variable est de type entier
- Pour une variable a de type float, le type adresse (pointeur) est noté : float *,
-

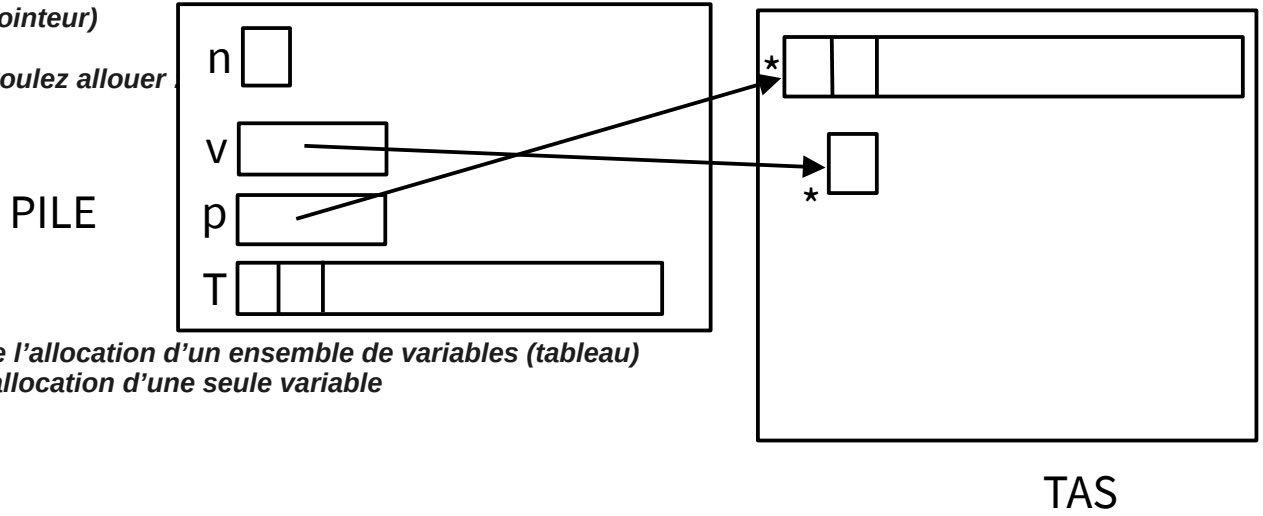
```
{
    Point A ;
    cout<< & A<<endl ;
}
```

  &A est de type Point * ;
- Ici & intervient en tant que opérateur (appliqué à une variable) et * intervient dans une déclaration de type

New et Delete

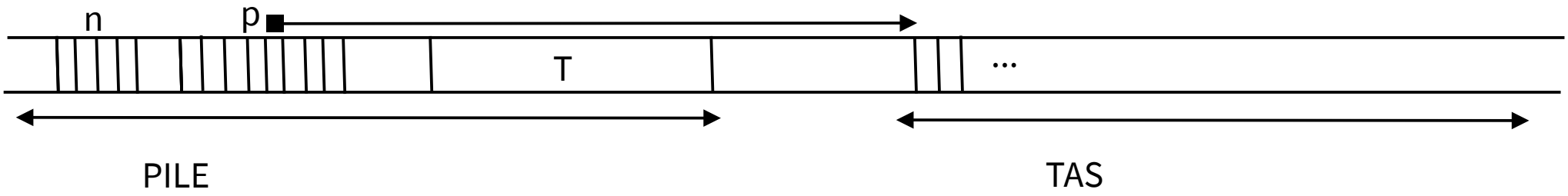
- L'allocation mémoire avec new se fait dans un le tas
- *{*

```
int *p ; // p est une variable de type adresse (pointeur)
int n ;
cout<<"Entrez le nombre d'entiers que vous voulez allouer
cin >> n ;
p=new int[n] ;
....
....
int *v ;
v=new int ;
...
int T[100] ;
delete [] p ; // avec des crochets car il s'agit de l'allocation d'un ensemble de variables (tableau)
delete v ; // pas des crochets car il s'agit de l'allocation d'une seule variable
}
```



- p est une variable de type adresse sur entier
- Je peux allouer un emplacement (dans le tas) pour n entier : p=new int[n] ; new renvoie une adresse où est réservé l'emplacement mémoire
- p est maintenant un tableau d'entiers
- Je peux aussi allouer un seul entier : v=new int ;
- A la fin du bloc : l'espace alloué (dans la pile) avec « int T[100] » est restitué au système, contrairement à celui alloué dans le tas avec « p=new int[n] »
- Pour désallouer l'espace mémoire réservé avec « *p=new int[n] ; » il faut le faire explicitement avec « delete [] p »*
- *Pour désallouer* l'espace mémoire réservé avec « v=*new int ; » il faut le faire explicitement avec « delete v »*

La mémoire est unidimensionnelle



Allocation, classe

- #include <iostream>

```
class Point
{
   int x ;
   int y ;
} ;
int main()
{
   Point *p ; // un pointeur sur une variable de type Point
   p=new Point ; // j'alloue dynamiquement une variable de type Point
   cout<< "("<<p→x<<","<<p→y<<")"<<endl ;// Comme p est une adresse (un pointeur)
   // ci dessus j'utilise → à la place de « . »
   delete p ; // je libère la mémoire allouée pour.
   Point *t ; // un pointeur sur une variable de type Point : le point n'existe pas encore
   int n=300 ;
   t=new Point[n] ; // j'alloue dynamiquement (dans le tas) un espace mémoire pouvant contenir 300 Points)
   t[62].x=3 ; t[62].y=3 ; // t est un tableau de points
}
```

Les rôles de * et de &

- **Dans une déclaration (dans une assignation de type)**
  - int *p ; // p est un pointeur vers un entier
  - Point *p ;// p est est un pointeur vers un point
  - int & a ; // a est une variable de type entier passée ou récupérée par référence (ou par variable)
  - Point & a ; // a est une variable de type Point passée ou récupérée par référence
- **En tant qu'opérateur**
  - L'opérateur & s 'applique à une variable
    - *int x=5 ; cout<<(long) &x<<endl ; // & retourne l'adresse d'une variable (c'est une valeur de type pointeur)*
    - *int *p=&x ; // * intervient ici pour assigner le type pointeur à la variable p : p et &x ont le même type*
    - *Point P ; cout<<(long) &P<<endl ; // & retourne l'adresse d'une variable (c'est une valeur de type pointeur)*
    - *Point *a=&P ; // * intervient ici pour assigner le type pointeur à la variable a : a et &P ont le même type*
  - L'opérateur * s'applique à une adresse (un pointeur)
    - *cout<< *p<<endl ; //affiche 5, la valeur contenue dans la variable pointés par p c'est-à-dire x*
    - *a c'est P : *a c'est l'objet pointé par a, c'est-à-dire P

Exemple1

Ecrire une fonction qui :

- Demande à l'utilisateur une valeur entière n

(correspondant à la taille du tableau qu'il veut construire), cette valeur doit être renvoyée à par la fonction

- Elle doit créer dynamiquement un tableau de n entiers

- Elle doit remplir le tableau de valeurs aléatoires entre 0 et 100

```
int * Creer_et_remplir(int & n)
{
   cout<< "Entrez n :" ;
   cin>>n ;
   int *r=new int[n] ;
   for(int i=0;i<n;i++) r[i]=rand()%101 ;
   return (r) ;
}

void afficher(int *T, int n)
{
   for(int i=0;i<n;i++) cout<<T[i]<<" " ;
   cout<<endl ;
}

}
```

```
int main()
{
     /* Créer la variable taille pour la donner à
     la fonction Creer_et_remplir */

     int taille;

     /* Créer la variable T de type pointeur pour
      récupérer l'adresse renvoyé par la
       fonction Creer_et_remplir*/

     int *T=Creer_et_remplir(taille) ;
     afficher(T,taille) ;

     /* à partir de maintenant T est un tableau
       d'entiers */
     …...
}
```

Exemple2

Ecrire une fonction qui prend en argument un tableau de Points et sa taille et qui renvoie un tableau de Points crée dynamiquement sur mesure contenant tous les points dont l'abscisse est supérieure à l'ordonnée. N'oubliez pas qu'il faut retourner la taille du nouveau tableau.

```
Point * Selection_Points(Point *T, int N, int &taille_selection)
{
  taille_selection=0 ;

  for(int i=0;i<N;i++) if(T[i].x >= T[i].y) taille_selection=taille_selection+1 ;

   if(taille_selection>0)
   {
      Point *R=new Point[taille_selection] ;
      int k=0 ;
      for(int i=0;i<N;i++) if(T[i].x >= T[i].y) {R[k]=T[i] ; k++;}
      return R ;
   }
   else return( NULL) ;

   }
```

Exemple 3

f- Écrire une fonction qui prend en argument un tableau d'entiers et sa taille et qui renvoie 2 tableaux d'entiers crées dynamiquement sur mesure contenant d'une part toutes les valeurs positives et d'autre part toutes les valeurs négatives. N'oubliez pas qu'il faut retourner les tailles des nouveaux tableaux. Attention on ne peut pas allouer un tableau de taille 0.

Correction f :

Voici le prototype

```
void Selection_Points(Point *T, int N, int * & T1, int * & T2, int & taille1, int & taille2)
```

T1 et T2 sont deux pointeurs passés par variable

```
T1=new int[taille1] ;
T2=new int[taille2] ;
```

Constructeur / Allocation Dynamique

- Rôle du constructeur :
    - initialisation des valeurs des attributs
    - compléter la création en allouant de l'espace mémoire (new)

Exemple :

```
class ens_Points
{
    Point *T ;
    int cap ; //un entier indiquant la capacité max de l'ensemble
    int nbp ; // un entier indiquant le nombre de points réellement stockés
    ens_Points(int cap_=100) ; // un constructeur
    ~ens_Points()
    …..
}
```

Constructeur / Allocation Dynamique

- le constructeur est appelé juste après la création de l'objet. A ce moment le pointeur T ne pointe sur rien.

- Dans le fichier ens_Points.cpp :

```
ens_Points::ens_Points(int cap_=100)
{
     nbp=0 ;
     cap=cap_ ;
     if(cap_>0)
          T=new Point[cap_] ;
     else
     {
          cout<<"la capacité doit être positive"<<endl ;
          exit(1) ;
     }
}
```

– Le Destructeur est appelé juste avant la disparition de l 'objet

```
ens_Points::~ens¨points()
{
     delete T ;
}
```

– Et s'il n y avait pas de destructeur !!!

⇒ l'objet disparaitra, mais l'emplacement mémoire sur le quel T pointait existerait toujours

⇒ fuite mémoire

Tableau à deux dimentions

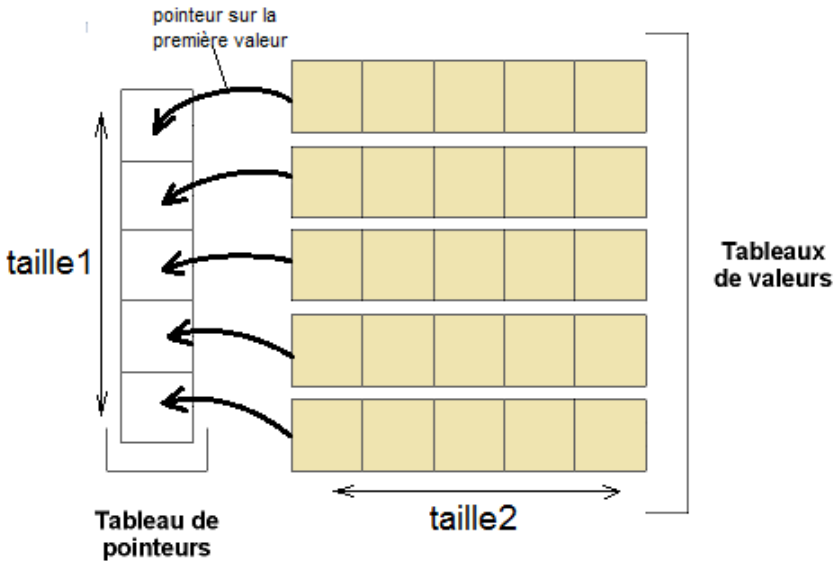
- Pour allouer le tableau de la figure :

- 1 - allouer le tableau de pointeurs :

```
Point * *T=new Point * [taille1] ;
```

- 2 – pour chaque case T[i], on alloue un tableau de points :

```
for(int i=0;i<taille1;i++)
    T[i]=new Point[taille2] ;
    // T[i] est de type Point *
```



– Pour libérer le tableau de la figure :

- 1 – libérer tous les T[i]

```
for(int i=0;i<taille1;i++)
    delete [] T[i] ;
```

- 2 – libérer T :

```
Delete [] T ;
```

La classe Image niveau de gris

```
class ImageGris
{
private:
    int **pixels;   // matrice 2D de pixels (0 à 255)
    int hauteur;
    int largeur;
public:
   ImageGris(int h, int l) ;
   void setPixel(int i, int j, int valeur) ;
   void afficher() ;
   ~ImageGris() ;
}
ImageGris ::~ImageGris()
{
        for (int i = 0; i < hauteur; i++)
                    delete[] pixels[i];
        delete[] pixels;
        cout << "Mémoire de l'image libérée." << endl;
}
```

```
ImagGris ::ImageGris(int h, int l)
{
        hauteur = h;
        largeur = l;

        // Allocation du tableau de pointeurs (lignes)
        pixels = new int*[hauteur];

        // Allocation de chaque ligne
        for (int i = 0; i < hauteur; i++) {
            pixels[i] = new int[largeur];
        }

        // Initialisation : image noire (tous les pixels = 0)
        for (int i = 0; i < hauteur; i++) {
            for (int j = 0; j < largeur; j++) {
                pixels[i][j] = 0;
            }
        }
        cout << "Image " << hauteur << "x" << largeur << " créée." << endl;
}
```

Le constructeur par recopie

- Le constructeur par recopie est un constructeur spécial qui permet de créer un nouvel objet à partir d'un objet existant de la même classe.

- Le prototype du constructeur par recopie :

Point ::Point (const Point & p) ;

- Utilisation du constructeur par recopie

Point P(1,2) ;

Point Q(P) ; // on crée un nouvel objet Q qui est une copie de P, mais complètement distinct

Point Q=P ; // c'est pareil : ici « = » n'est pas un opérateur d'affectation

Q.afficher() ; // ⇒ (1,2), comme P.afficher()

- Les données membres de P et de Q sont physiquement distincts

P.setX(2) ; P.afficher() ; Q.afficher ; // ⇒ (2,2)(1,2) : les données membres sont différentes

Le problème en cas d'allocation dynamique et sans constructeur par recopie

```
class Tableau
 {
  private:
      int* tab;
      int taille;
      int nbe ;
  public:
      Tableau(int n)
      {
          taille = n;
          nbe=0 ;
          tab = new int[taille];
      }
     ~Tableau()
     {
        delete[] tab;
     }
};
```

- Tableau t(6) ;
  Tableau copie_t(t) ;
  // appel du constructeur de copie PAR DÉFAUT
  ⇒ copie_t.tab=t.tab et
  copie_t.taille=t.taille
- Conséquence grave :
  + Les deux objet t et copie_t pointent vers le même tableau
  + Quand t2 est détruit → delete[] tab
  + Puis quand t1 est détruit → double delete ⇒ crash du programme

Solution : écrire un constructeur de copie (copie profonde)

```
class Tableau
{
private:
    int* tab;
    int taille; // la taille
    int nbe ; // le nombre d'éléments

public:
    Tableau(int n) ;
    // CONSTRUCTEUR PAR RECOPIE (IMPORTANT)
    Tableau(const Tableau& autre) ;
    ~Tableau() ;
};
```

```
Tableau::Tableau(int n)
{
     taille=n ;
     nbe=0 ;
     tab=new int[n] ;
}
Tableau::Tableau(const Tableau &a)
{
   taille = a.taille ;
   nbe = a.nbe ;
   tab = new int[taille] ;
   for (int i=0;i<taille;i++) tab[i]=a.tab[i] ;
}
```

Bonnes habitudes

- **Passez toujours les objets en argument par référence**

  Quitte à utiliser const pour protéger les objet si nécessaire

- **Toujours un destructeur**

  si les objets contiennent des parties dynamiques

- **Toujours quelques constructeurs**

  facilitant l'utilisation de votre classe

- **Toujours un constructeur par recopie**

- **Toujours l'opérateur d'affectation (voir page suivante)**

L’opérateur d’affectation

```
Tableau& Tableau ::operator=(const Tableau& autre)
{
    // 1) Vérifier l’auto-affectation
    if (this == &autre)
        return *this;

    // 2) Libérer l’ancienne mémoire
    If (tab) delete[] tab;

    // 3) Allouer une nouvelle zone mémoire
    taille = autre.taille;
    tab = new int[taille];

    // 4) Copier les valeurs
    for (int i = 0; i < taille; i++) {
        tab[i] = autre.tab[i];
    }

    // 5) Retourner *this (convention C++)
    return *this;
}
```

***Cela permet de faire :**

```
Tableau A(10) ;
A.ajouter(2) ;
A.ajouter(5) ;
//Appel au constructeur par recopie
Tableau B=A;
Tableau C ;
// Appel l’opérateur d’affectation « = »
C=A ;
```